Evaluating LLMs as Cross-Auditors of LLM Generated Android Vulnerability Analysis

Toward Designing a Multi-Agent Framework for Auditing LLM-Guided Android Security Analysis

João R. Cardoso
University of Southern Denmark
MMMI
Odense, Denmark
jocar25@student.sdu.dk

Lily B. Wanscher
University of Southern Denmark
MMMI
Odense, Denmark
liwan21@student.sdu.dk

Denisa A. Rissa
University of Southern Denmark
MMMI
Odense, Denmark
deris22@student.sdu.dk

Yusuf M. Yusuf
University of Southern Denmark
MMMI
Odense, Denmark
yuyus19@student.sdu.dk

***Abstract*—This project investigated various LLM models' ability to audit their own, and others, findings in regards to security analysis, in order to find and quantify a level of trustworthiness. To do this, a framework for security analysis by any model able to be run through Ollama, was established, a dataset curated, a series of evaluation metrics defined (Technical Accuracy, Effect Chain Awareness, Hallucination Frequency, and Attack Surface Coverage), and a testing procedure defined and executed.**

The project found that although model performance varied greatly between subjects in the testing sample, Hallucination Frequency was independent from other defined metrics. Summarily, this means that a model may be quite apt and accurate, when not hallucinating. However, even the best performing model did not prove to be reliable to the point of being trustworthy.

I. Introduction

Hybrid android applications occupy a uniquely challenging position in mobile security. By enabling communication between a WebView and native Java or Kotlin code to the extents of arbitrary code execution, developers gain the ability to use existing web standards to manage and render content, but they also introduce an attack surface that is difficult to reason about for the individual developer, or analyze statically.

The central mechanism is `addJavascriptInterface`, an Android API that exposes Java objects and their methods directly to JavaScript running inside the WebView. The implications are significant: any JavaScript executing in that context can invoke arbitrary Java methods on the bridge, including operations that touch the file system, send network requests, query databases, or leak device identifiers. When those methods lack proper input validation, origin checking, or access control, the result is a class of vulnerabilities spanning both sides of the language boundary simultaneously, including path traversal, intent redirection, PII exposure, and SQL injection.

Large Language Models present a compelling complement to traditional static analysis tools in this space. Given a structured summary of bridge interfaces alongside recovered JavaScript snippets, an LLM can reason across the language boundary, infer semantic intent from naming conventions, trace plausible data-flow paths, and generate findings that a symbol-level static analyzer would miss. The obstacle is not capability, it is trust. LLMs are non-deterministic and prone to hallucination, producing confident-sounding findings that reference methods which do not exist, invent data-flow paths not present in the evidence, or overstate the severity of issues whose preconditions are never met. In a security context, this unreliability is not a minor inconvenience but a fundamental barrier to operational use.

This paper presents a multi-agent framework designed to quantify how much an LLM-generated security report can be trusted. The core idea is to separate analysis from verification: one set of LLM agents performs the security analysis, and a second set evaluates each output against the raw evidence, producing structured quality metrics rather than a binary verdict. For each analyzed callsite, the framework computes four metrics covering hallucination frequency, technical accuracy, effect-chain awareness, and attack-surface coverage, which combine into a composite Trustworthiness Score ranging from 0.0 to 1.0.

The remainder of this paper is structured as follows. It first reviews the state of the art in static analysis of Android hybrid applications and in multi-agent LLM frameworks, then establishes the theoretical concepts the framework rests on and how that theory shaped the prompt and agent design. It next describes the development process, the testing setup and model population, and how the work was organized, before presenting the results, discussing them against the research questions, concluding, and outlining directions for future work.

A. Research Questions

- **RQ A — Quantifying Trustworthiness.** What combination of metrics (e.g., hallucination frequency, technical accuracy, effect-chain awareness, and attack-surface coverage) best captures a meaningful *Trustworthiness Score* for LLM-based security auditing?
- **RQ B — Agent Benchmarking.** Using aforementioned trustworthiness metrics, how do different model architectures or model parameter counts compare when cross-verifying Java–JavaScript data-flow risks?
- **RQ C — Failure Taxonomy.** What is the nature of the weaknesses or shortcomings that each chosen Auditor model present - if any?

B. Hypotheses

- **Null Hypothesis (H_0).** The mean error-detection rate of the best-performing Auditor agent is less than or equal to that of a baseline random classifier ($p = 0.05$).
- **Alternative Hypothesis (H_1).** The mean error-detection rate of the best-performing Auditor agent is significantly greater than the baseline, demonstrating a real and measurable ability to catch hallucinations or logical errors in the primary analysis.

C. Contribution

This paper aims to contribute to existing literature by establishing a method for quantifying and analyzing how capable various models are of verifying their own, and others', work within the context of security analysis. One aspect is how good a large language model may be at producing functional code in terms of programmatic proficiency and language awareness, another is how capable the same, or another, model may be at analyzing and catching mistakes and vulnerabilities on their own.

In other words, this paper aims to find and prove how trustworthy certain models are.

D. Concessions

Testing population diversity was limited due to lack of compute and funding. As in, the largest model tested had but 14 billion parameters. This will be further detailed in Testing ¿ Population and presented as possible future work in Perspective.

II. State of the Art

A. Static Analysis of Android Hybrid Applications

The security challenges introduced by Android's WebView bridge mechanism have attracted considerable attention from the static analysis community. The central difficulty is that hybrid applications combine two execution environments, Java and JavaScript, whose interaction is difficult to reason about from either side alone.

LUDroid, proposed by Tiwari et al. [1], represents one of the most comprehensive large-scale studies of this problem. The framework performs information flow analysis on 7,500 real-world applications from the Google Play Store, extracting data flows from Android to JavaScript through the `addJavascriptInterface` and `loadUrl` APIs. LUDroid decompiles APKs using APKTool and applies backward slicing to identify the Java objects injected into WebView contexts, then categorizes the resulting flows as sensitive or benign based on established Android source-sink definitions. Its findings are significant: 68% of sampled applications used WebView, and among those, 82% of sensitive information flows could reach potentially untrusted JavaScript code. The study also identified 365 applications loading content over unencrypted HTTP, exposing them to man-in-the-middle substitution attacks, and found 653 applications where JavaScript transmitted data back to bridged Android objects, implementing a two-way communication channel that conventional one-directional taint analyses would miss entirely.

LUDroid's contribution is primarily empirical: it establishes what the WebView bridge is used for in practice and where the dangerous patterns concentrate. Its limitations are equally instructive. The framework does not follow the JavaScript side of the bridge dynamically; code fetched from remote servers or constructed at runtime falls outside its analysis scope. The authors explicitly note that any JavaScript loaded via unencrypted protocols, or injected from third-party libraries, is treated as a black box. This boundary precisely defines the gap that motivates the present work.

iWanDroid [2] addresses a complementary aspect of the same problem. Rather than focusing on information flow statistics, it provides structured extraction of WebView bridge interface definitions from real-world open source applications, drawing primarily from the F-Droid repository. The resulting dataset, which forms the evaluation corpus for this paper, encodes bridge interface declarations, exposed method signatures, and associated metadata in a form suitable for downstream analysis. Like LUDroid, iWanDroid operates statically and therefore cannot observe JavaScript behavior at runtime or reason about code loaded from remote origins.

B. Limitations of Static Approaches and the Role of Semantic Reasoning

Taken together, tools such as LUDroid and iWanDroid demonstrate that the Java side of the WebView bridge is tractable for static analysis: method signatures, injected class names, and data-flow paths through the Android codebase can be extracted at scale with reasonable precision. What neither approach can provide is semantic interpretation of what those interfaces imply in a concrete attack scenario. A method named `readFile` accepting a path parameter is structurally visible to static analysis but its exploitability under a path traversal attack requires reasoning about naming conventions, parameter semantics, and plausible attacker behavior that falls outside the scope of symbol-level tools.

This gap motivates the use of Large Language Models as an analytical layer on top of statically extracted bridge data. Recent work on LLM-assisted vulnerability detection has shown that frontier models can reason across abstraction

boundaries, infer intent from naming conventions, and generate coherent findings from structured program summaries [3]. The central challenge, as this paper addresses, is not whether such reasoning is possible but whether it is reliable enough to be trusted in an operational security context.

C. Multi-Agent Frameworks for LLM Reliability

A recurring finding across LLM-assisted security analysis is that single-model outputs are unreliable in isolation: they produce plausible but incorrect findings, and no internal signal distinguishes a hallucinated result from a well-evidenced one. Multi-agent architectures have emerged as a structural response to this problem, distributing generation and verification across distinct agent roles so that one model's output becomes another model's input for scrutiny.

The foundational motivation for this design comes from work on multi-agent debate in general reasoning tasks. Liang et al. [4] identify a *Degeneration-of-Thought* problem in self-reflective LLMs: once a model has committed to a position, iterative self-correction fails to dislodge incorrect conclusions. Their Multi-Agent Debate (MAD) framework addresses this by having multiple agents argue opposing positions under a judge agent, demonstrating that inter-agent disagreement surfaces errors that single-model reflection cannot. Crucially, they also find that using different LLMs as agents introduces fairness problems in adjudication, a concern directly relevant to multi-model auditing designs.

This debate principle has been applied directly to security analysis. GPTLens [5] proposes an adversarial two-stage framework for smart contract vulnerability detection in which an auditor agent generates candidate vulnerabilities broadly, and a critic agent then evaluates each finding for correctness, severity, and exploitability. The separation of generation from discrimination significantly reduces false positives compared to single-stage detection, while preserving recall. However, GPTLens uses the same underlying model for both roles, which limits the independence of the critique: the critic inherits the same biases and blind spots as the auditor. The Trust, but Verify framework extends this principle by decoupling the two roles across different models entirely, allowing the Auditor's assessment to be genuinely independent of the Inspector's assumptions.

More recent work reinforces both the promise and the limits of the approach. Sheng et al. [3] identify reliability and verification of model outputs as central open problems in LLM-based security analysis, noting that false positive rates remain high and that incorrect reasoning can accompany correct vulnerability identification. This compounds the challenge: an audit layer must evaluate not only whether a finding is present in the evidence, but whether the reasoning chain connecting evidence to conclusion is sound. The Trustworthiness Score defined in this paper operationalizes exactly that distinction through its separation of hallucination frequency from technical accuracy and effect-chain awareness.

III. Theoretical Concepts

A. Hybrid Android Applications and WebView

Hybrid Android applications combine native Android code with web-based content rendered through WebView. In this paper, the important feature of this architecture is not the use of web content itself, but the presence of two interacting execution environments: Java or Kotlin on the Android side and JavaScript inside the WebView.

This distinction matters because each environment operates under different security assumptions. Native code may access Android resources such as files, intents, databases, network APIs, and device identifiers, while JavaScript is usually constrained by the web context in which it executes. Security issues arise when mechanisms exist for JavaScript to reach native functionality [6].

B. Java–JavaScript Bridges

A Java–JavaScript bridge is the interface through which JavaScript inside a WebView can invoke native Android methods. Android commonly provides this functionality through `addJavascriptInterface`, which exposes a Java or Kotlin object to the JavaScript environment under a chosen interface name [6].

For this work, the bridge is treated as a security boundary rather than just an implementation mechanism. An exposed method must be evaluated not only by its native implementation, but also by whether it is reachable from JavaScript, whether its inputs can be attacker-controlled, and what downstream effects it can trigger. This bridge-centered view motivates the callsite-level analysis used in the framework.

C. Prompt Engineering

Prompt engineering refers to the practice of designing and refining inputs to language models in order to elicit accurate, relevant, and well-structured outputs. Rather than modifying model weights or fine-tuning on domain-specific data, prompt engineering works within the inference-time interface, shaping model behavior through the construction of the input itself [7].

In the context of this work, prompt engineering is the primary mechanism through which the analysis framework communicates tasks to the language model. Each prompt encodes not only the question to be answered, but also the relevant context, the expected output format, and any constraints the model should respect. Because the model has no persistent state between invocations, every prompt must be self-contained, carrying all information necessary for the model to produce a useful response.

Several techniques inform the prompts used in this framework. Chain-of-thought prompting encourages the model to produce intermediate reasoning steps before arriving at a conclusion, which has been shown to improve performance on multi-step tasks [**?**]. Structured output prompting constrains the model to respond in a machine-parseable format such as JSON, allowing downstream components to consume results programmatically without fragile text parsing. Few-shot prompting provides the model with representative examples

of input-output pairs, reducing ambiguity and anchoring the model to the expected response style.

Prompt design in security analysis contexts introduces additional considerations. The model must be guided to reason about attacker-controlled inputs, trust boundaries, and reachability conditions, concepts that require precise framing to avoid superficial or incomplete assessments. Poorly constructed prompts risk producing outputs that are syntactically valid but analytically shallow, which motivates treating prompt construction as a first-class engineering concern within the framework.

IV. Applying Theory to the Design

A. Prompt Engineering for Inspector and Auditor Agents

Prompt engineering plays a central role in this framework because the models are not asked to produce open-ended security commentary. Instead, each model is assigned a defined role, a fixed evidence context, and a required output structure. This reduces ambiguity and makes outputs easier to compare across different models.

The framework uses a callsite-level unit of analysis rather than an application-level one. Each callsite corresponds to a single WebView bridge row and its associated JavaScript snippets. This narrows the task from broad application auditing to a focused evaluation of one Java-JavaScript bridge interaction.

The Inspector prompt casts the model as a security researcher analyzing a single Android WebView bridge callsite. The provided context includes the application package, Java bridge interface, exposed bridge methods, initiating method, and JavaScript snippets that serve as entry points. The model is then asked to evaluate the callsite using the five criteria shown in Table I.

TABLE I
Evaluation Criteria

Category	Description
A. Data Flow & Sink Analysis	Traces variables from JavaScript calls to Java sinks. Flags methods involved in SQL queries, file paths, intent creation, and dangerous execution mechanisms such as `Runtime.exec()` or reflection.
B. PII & Permission Leakage	Identifies Java methods that return sensitive data, such as `getDeviceId()`, `getAccounts()`, `getLocation()`, and `getSimSerialNumber()`. Checks whether JavaScript stores PII in `localStorage` or transmits it externally.
C. Reachability & Attacker Control	Determines which bridge methods are reachable by attacker-controlled JavaScript and how much input to each method is attacker-controlled.
D. Impact Chain & Privilege Escalation	Evaluates the worst-case outcome of invoking exposed methods with arbitrary input and whether multiple calls can be chained to escalate privileges.
E. Protocol & Origin Security	Checks whether content is loaded over `http://` and whether `message.origin` is properly validated in event listeners.

This prompt design encourages the model to ground its reasoning in the supplied evidence rather than relying on general Android security knowledge. To ensure consistency, each identified vulnerability must include a title, risk level, data-flow path, technical description, and supporting evidence. If no vulnerabilities are found, the model returns a standardized "No vulnerabilities identified" response.

The Auditor prompt serves a different purpose. Rather than generating new findings, it evaluates the Inspector's output against the same ground-truth context. The Auditor receives the bridge interface, JavaScript snippets, and Inspector findings, then returns a JSON object with four scores: hallucination frequency, technical accuracy, effect-chain awareness, and attack-surface coverage.

This distinction reflects the core principle of the framework: vulnerability discovery and vulnerability verification should be treated as separate tasks. The Inspector generates possible vulnerability claims, while the Auditor checks whether those claims are supported by the analyzed callsite.

Hallucination is treated as an evidence-grounding failure. A finding is considered hallucinated if it references a method, class, object, code segment, or behavior that does not appear in the provided context. A vulnerability claim is therefore only trustworthy if it can be traced back to the bridge methods, JavaScript snippets, or other callsite evidence.

B. Structured Output and Chain-of-Thought Scaffolding

Two prompting techniques from the theoretical background are applied in concrete ways in the Inspector and Auditor prompts.

The Inspector prompt uses the five evaluation criteria as an explicit reasoning scaffold. Rather than asking the model to find vulnerabilities freely, it directs the model through a fixed sequence: data flow and sink analysis, PII and permission leakage, reachability and attacker control, impact chain and privilege escalation, and protocol and origin security. This structure functions as a form of chain-of-thought prompting [7]: the model is required to work through each category before producing findings, which reduces the likelihood of superficial outputs that skip attacker reachability or downstream impact. A finding that omits a data-flow path is structurally invalid under the required output format, which creates an additional incentive for the model to reason through the full chain.

The output formats for both agents enforce structured output prompting directly. The Inspector prompt mandates a strict Markdown template for each finding, requiring a title, risk level, data-flow path, technical description, and quoted evidence. A standardized fallback response is required when no vulnerabilities are identified, preventing free-form negative outputs that would break downstream parsing. The parser in `utils.py` splits the response on `### Finding:` headers using a regular expression, producing a list of self-contained

finding blocks that can be passed independently to each Auditor.

The Auditor prompt takes a stricter approach. The expected JSON schema is embedded directly in the prompt text, and the model is instructed to return only valid JSON with no Markdown fences or surrounding commentary. The parser applies type coercion after extraction, converting hallucination frequency and attack-surface coverage to floats and technical accuracy and effect-chain awareness to integers according to the schema defined in `_METRICS_TYPES`. A fallback strips Markdown fences defensively in case a model includes them despite the instruction. This combination reflects the design priority of making outputs machine-consumable without post-hoc human correction.

C. Multi-Agent Architecture and Cross-Audit Design

The framework implements the generation-discrimination separation principle described in GPTLens [5], but extends it in two important ways. First, the Inspector and Auditor roles are filled by different model instances rather than the same underlying model within the same session. A model auditing its own output inherits the same biases and blind spots that produced that output, if done within the same session context, using distinct model instances introduces genuinely independent priors. This directly addresses the fairness concern raised by Liang et al. [4] regarding multi-agent adjudication: an auditor that was not the originating inspector has no prior commitment to the correctness of the findings it is evaluating.

Second, the framework does not pair each Inspector with a single dedicated Auditor. Every model in the configuration serves as both an Inspector and an Auditor. For a configuration of N models, this produces N sets of Inspector findings and $N \times N$ audit evaluations per callsite. Each Inspector's output is therefore evaluated by all N Auditors independently, including itself. The resulting audit matrix provides multiple independent quality signals per finding set, which is necessary given the near-zero ICC values observed in the results: individual audit scores are noisy, and only aggregated means across multiple Auditors are reliable enough to cite with confidence.

Execution is organized into two sequential phases. In the first phase, all Inspector models are run across all callsites before any Auditor work begins. After each Inspector model finishes, it is explicitly unloaded from GPU memory via the Ollama API (`keep_alive=0`), freeing resources for the next model. This sequential-per-model strategy was chosen because running multiple large language models concurrently on a single consumer GPU is not feasible within the compute constraints of this project. In the second phase, each Auditor model processes all callsites and all Inspector outputs, then is unloaded in turn. The two-phase separation ensures that Auditors always operate on complete Inspector outputs rather than partial results, and it keeps peak memory usage bounded to a single model at a time.

The intermediate state between phases, Inspector findings indexed by callsite and model identifier, is held in memory and written to disk incrementally as each callsite record is finalized. This design tolerates interruption: if execution is halted between callsites, the output file contains all records up to that point in a valid JSON array, and the pipeline can resume from the last written index without reprocessing completed callsites.

V. Development Process

The development process followed six sequential phases: literature review, problem and metric definition, prompt and architecture design, implementation, informal testing, and full pipeline execution.

The literature review informed both the choice of evaluation corpus and the multi-agent architecture. The iWanDroid dataset [2] was selected as the primary input because it provides structured bridge interface definitions suited to callsite-level analysis. The dataset produced 145 callsite entries in total, all of which were processed by the pipeline, drawn as evenly spaced groups across the full array. These spanned 41 distinct applications. Manual validation was carried out on a subset of callsites within the available time.

With the scope established, the four evaluation metrics were defined before any prompts were written: Hallucination Frequency, Technical Accuracy, Effect-Chain Awareness, and Attack-Surface Coverage. Fixing the metrics early meant that the Inspector and Auditor prompts could be designed around concrete output targets rather than adjusted post-hoc to fit whatever the models happened to produce.

Prompt design followed the metric definitions. The Inspector prompt assigns the model a specific role, supplies all callsite evidence inline, and requires findings to follow a fixed Markdown template covering data-flow paths, risk levels, and quoted evidence. The Auditor prompt supplies the same callsite context alongside the Inspector's output and requires a JSON object keyed to the four metrics. Both prompts were initially constructed to reflect established prompt engineering practices, including chain-of-thought scaffolding for the Inspector and strict structured output constraints for the Auditor [7].

The implementation used Python as the primary language, with Ollama managing local model serving and GPU memory. Each model is loaded, run to completion across all assigned callsites, and then explicitly unloaded via the Ollama API before the next model is started. Intermediate state is written to disk incrementally as each callsite record is finalized, allowing the pipeline to resume from the last completed index without reprocessing. Output is stored in JSON for per-record structure and CSV for downstream aggregation; a SQLite database holds the full result set for querying during analysis.

The two-phase execution order, all Inspectors completing before any Auditor work begins, was not a deliberate architectural choice at the outset. It emerged from the constraint that running multiple large language models concurrently on a single consumer GPU is not feasible. Sequential-per-model execution with explicit memory release between runs was the only practical strategy within the available compute, and the two-phase structure followed from it.

Informal testing against a subset of the samples preceded full execution. Prompt revisions were triggered by all four categories of failure: JSON parsing errors from non-compliant Auditor output, Inspector findings that referenced methods absent from the supplied context, reasoning that identified a vulnerability without tracing its downstream effects, and output format violations that broke the Markdown parser. Each revision was applied uniformly across both prompts before re-testing. The prompts used in the final pipeline execution reflect the state reached after this iterative process concluded.

VI. Testing

A. Metrics

In order to evaluate the abilities of a model for the purposes at hand, a series of metrics were defined. These metrics were defined based on what aspects of model performance were deemed relevant given the context.

- Hallucination Frequency (HF) [%]:
  How many times did Inspector N hallucinate findings compared to the inspector who hallucinated the most. Requires definition of 'Hallucination', i.e. makes up findings or code segment that does not exist in the evaluated case
- Technical Accuracy (TA) [1..10]:
  Upon any finding, how technically correct is the Inspector in its description of the vulnerability
- Effect Chain Awareness (ECA) [1..10]:
  For a given identified vulnerability how aware is the Inspector of the possible chain of effects made possible by this vulnerability
- Attack Surface Coverage (ASC) [%]:
  How many vulnerabilities did the Inspector identify compared to researcher evaluation

B. Population

With the testing metrics defined, the testing population was identified. Given limitations as to available compute and funding, larger established models were not available, however any model which could run through Ollama was. This placed a fundamental limit on the diversity achievable in the testing population, especially in terms of model size but specifically in terms of parameter count. Within this limit, the sample was designed to consist of models with different parameter counts, architectures, and providers. The final identified population became:

Smaller models:
- llama3.2:3b
- qwen2.5-coder:7b
- starcoder2:7b

Medium models:
- deepseek-r1:8b
- gemma3:12b
- phi4:14b

According to K. Sun and R. Wang 2026 [8] the correlation between parameter count and model performance, is rather complex however, thus the upper bound of parameter count among selected models of but 14 billion does not inherently void the testing samples applicability to the generalized population.

C. Storage & Sampling

Before testing started, a JSON structure was designed to keep track of all information involved, generated, and added, for each callsite identified in the IWanDroid dataset. This was done to assure that no information was lost and that the findings would be easily processable when testing concluded.

All in all the dataset generated 145 such structures, spanning 41 distinct applications, all of which were processed through the Inspector and Auditor passes. Manual ground-truth conclusions were then recorded by researchers for 73 of these callsites, with the assigned entries drawn as evenly spaced groups throughout the array. This meant some randomness, but also that more than one sample appeared for the same app at times, each regarding a different callsite within said app.

D. Procedure

The aforementioned JSON structures first contained the models to run as inspectors, the models to run as auditors, the callsite location, package name, and other metadata concerning the application regarded. Later, the structures where appended the inspectors findings in one pass of processing, and then the auditors' results in another pass. Lastly, each researcher was assigned a group of entries to complete by appending the quantified ground truth in a "researcherConclusion" field by regarding each inspector's findings, against the auditors conclusion as to the inspector's performance, against the source code for the application.

E. Post-Processing

Qwen2.5-coder:7b and starcoder2:7b exhibited a behavior of repeating the same series of findings for a given identified callsite many hundreds of times. This led to a series of postprocessing passes being applied to detect repeated findings and replacing it with a corresponding "last N lines repeated M times". Thus making the data easier to work with for the researchers while not losing any information as the to performance of each Inspector.

F. Results

All results were stored in a pre-defined JSON format for easier processing. This segment details the performance differences between the models tested against ground truth as set by the researchers.

VII. Development Work Organisation

A team of four ran the project, coordinating through a small set of shared tools rather than a formal management framework. Day-to-day communication ran over a dedicated Discord server, which carried design discussion, status updates, and the small decisions that shaped the pipeline as it came together. Task tracking happened on a Trello board, with work split into packages and pushed across columns from open to done. The analysis code sat in a shared GitHub repository under the group organisation; version control gave every member the pipeline, the prompt definitions, and the full result sets.

Responsibilities were divided by area rather than spread evenly over one identical task list. Metric definition was the single piece every member shaped, because the four scoring dimensions had to be agreed before any prompt or audit could be built around them. Individual members then took separate strands: system architecture and prompt engineering, test-suite execution and result processing, the candidate model list, and the integration work joining the components. Manual validation of the model output demanded the most effort overall, so the callsites were partitioned into batches and shared out for parallel annotation.

VIII. Results

A. Dataset and Coverage

The pipeline execution covered 145 callsites spanning 41 distinct Android applications from the iWanDroid corpus. The six models described in the previous section each acted as both Inspector and Auditor, processing every callsite to produce a $6 \times 6 = 36$ cell audit matrix that across the full population amounted to 5,220 audit cells in total. Of these, 4,189 contained scored output; the remaining empties were dominated by starcoder2:7b, which returned no findings on 136 of the 145 callsites it was given, leaving Auditors with nothing to evaluate on its rows of the matrix.

Inspector productivity varied by an order of magnitude throughout the lineup. qwen2.5-coder:7b produced 883 findings, followed by llama3.2:3b at 588, gemma3:12b at 555, phi4:14b at 427, and deepseek-r1:8b at 244. Far below sat starcoder2:7b at 27.

Where Inspectors returned no findings at all, the same ordering holds. gemma3:12b engaged with every callsite in the population; qwen2.5-coder:7b returned empty on 2 of 145; phi4:14b on 21; llama3.2:3b on 52; deepseek-r1:8b on 58; starcoder2:7b on 136. Auditor coverage on the remaining cells was comparatively even, with each Auditor filling between 679 and 711 of the 870 cells assigned to it.

All metric values reported in this section are Auditor-assigned: each Inspector's output on a callsite is scored by the Auditor models on the four metrics, and the figures below aggregate those Auditor scores across the population.

B. Metric Structure (RQ A)

Pairwise correlations between the four normalised metrics are shown in Figure 1. TA, ECA, and ASC covary strongly with one another: Pearson coefficients are 0.91 between TA and ECA, 0.72 between ECA and ASC, and 0.69 between TA and ASC. The Spearman version attenuates these magnitudes slightly without changing the pattern; the corresponding rank coefficients are 0.84, 0.79, and 0.74. By comparison, $1-HF$ does not correlate meaningfully with any of the other three; its coefficients sit between -0.10 and 0.15 and switch sign between Pearson and Spearman in two of the three cells.

Figure 2 presents a principal component analysis over the same four-dimensional space. PC1 captures approximately 64% of the total variance and loads roughly equally on TA, ECA, and ASC, contributing close to zero on $1-HF$. A

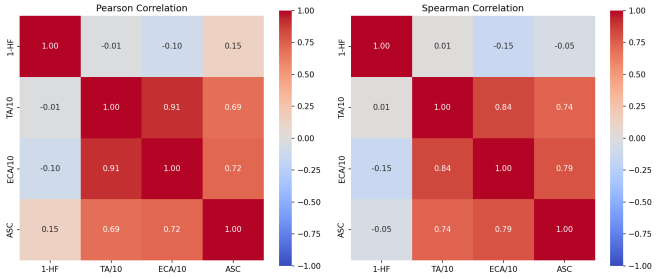

Fig. 1. Pearson and Spearman correlations between the four normalised metrics across all 145 callsites.

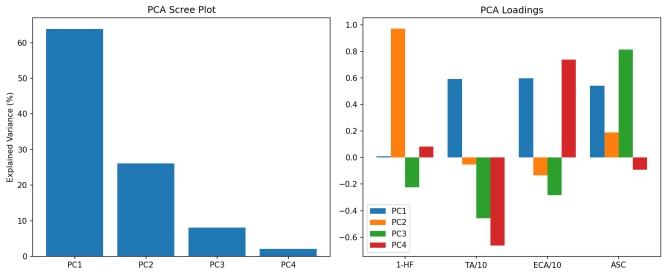

Fig. 2. PCA scree plot and component loadings over the four normalised metrics.

further 26% lands on PC2, which is loaded almost exclusively by $1-HF$. PC3 and PC4 together account for under 10% and add no separable structure on top of the first two. The four metrics reduce to two effective dimensions: a shared quality axis carried by TA, ECA, and ASC, and a separate hallucination axis.

C. Model Performance (RQ B)

Figure 3 gives per-model audit-derived means on the four metrics, z-scored within column and annotated with raw values. gemma3:12b records the highest scores on TA, ECA, and ASC at 6.50, 5.50, and 0.52 respectively. phi4:14b matches it on TA at 6.67 but trails on ECA at 4.83 and ASC at 0.43. deepseek-r1:8b and qwen2.5-coder:7b form a further cluster behind that pair, with respective means of $(6.21, 4.67, 0.33)$ and $(5.38, 4.46, 0.35)$. llama3.2:3b sits below the cluster on every positively-oriented metric at $(4.50, 3.79, 0.22)$. On every metric, starcoder2:7b records the lowest values at $(1.29, 0.90, 0.05)$; its low HF of 0.14 follows from the small number of findings it produced rather than from restraint.

The composite Trustworthiness Score $\mathrm{TS}_{\mathrm{equal}}$, computed as an equal weighting of the four normalised metrics on a $[0, 1]$ scale, is plotted per callsite per model in Figure 4. gemma3:12b is at the top again, holding both the highest median and the tightest interquartile range. phi4:14b, deepseek-r1:8b, and qwen2.5-coder:7b overlap to a degree that any within-cluster ordering depends on the subsample considered. llama3.2:3b drops below them, and starcoder2:7b occupies a band of its own with most of its distribution compressed around 0.25. Within-model spread is widest for qwen2.5-coder:7b and llama3.2:3b, each covering roughly $[0.10, 0.85]$.

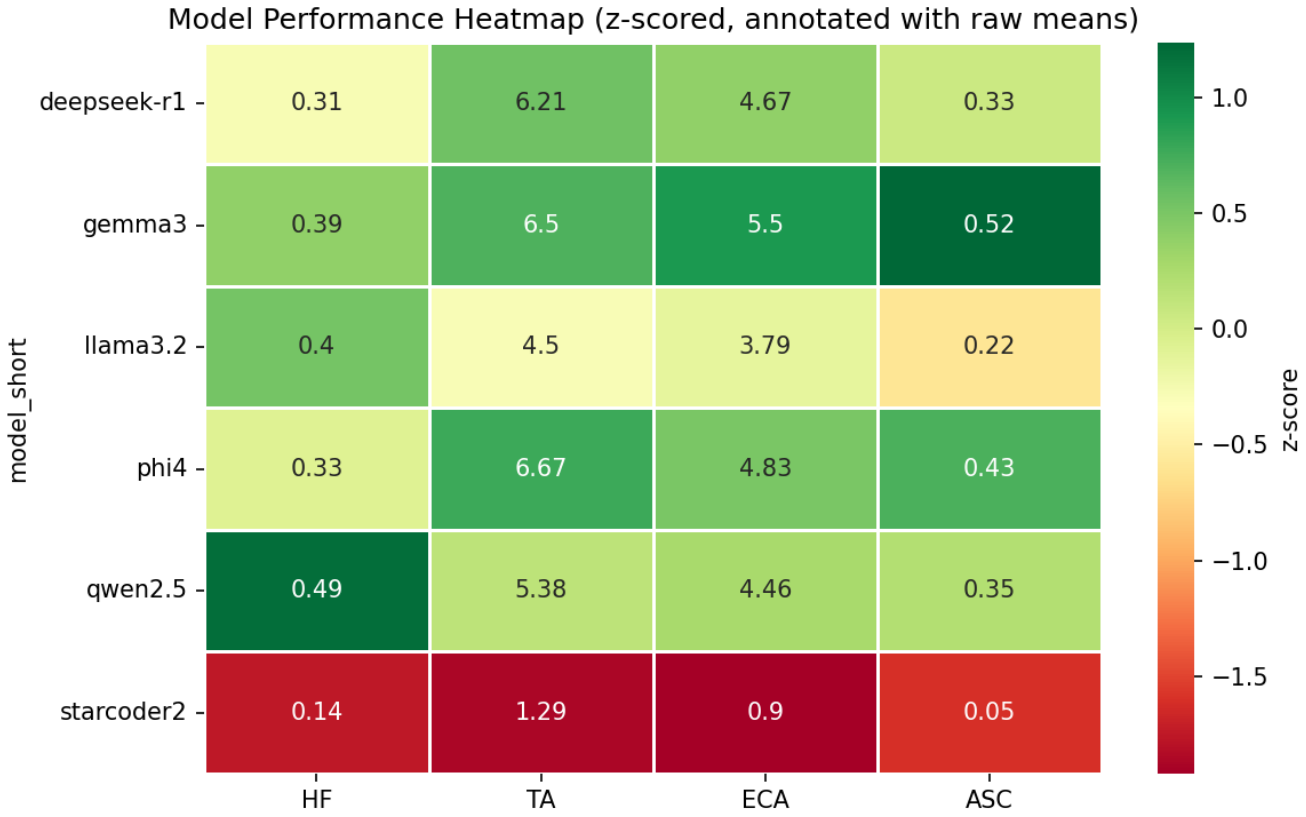

Fig. 3. Mean per-model metric values. Cells are z-scored within column; raw means are annotated.

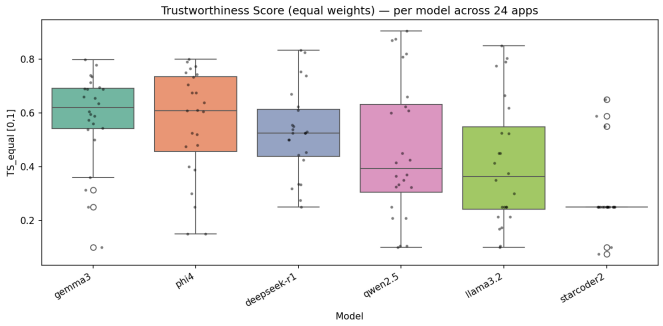

Fig. 4. Per-callsite TS_{equal} distribution per Inspector model.

Figure 5 shows the same means as a radar projection, making the model shapes directly comparable. gemma3 and phi4 trace nearly congruent polygons; deepseek-r1 closes them on TA and ECA but trails on ASC; qwen2.5 covers similar area to deepseek with a different profile; llama3.2 shrinks toward the centre on every axis except $1-HF$, where it remains comparable to the leading models; starcoder2 collapses toward the centre on all four axes.

The cross-batch view in Figure 6 reports mean TS_{equal} per model within each of the four evenly-spaced callsite batches drawn during sampling. The per-model ranking is preserved in every batch, and within-model variation stays bounded by approximately 0.05.

D. Failure Patterns (RQ C)

Figure 7 plots each of the four metrics against the context length supplied to the model. No metric shows a meaningful trend across the 0–3500 character range present in the dataset; Pearson coefficients stay below 0.13 in absolute terms for all four. Longer or richer context does not predict performance at the resolution available here.

A per-callsite view of TS_{equal} over the full population appears in Figure 8. Variation among rows within a column is often larger than variation among columns within a row; vertical bands of uniformly low scores mark callsites on which every model scored poorly, and isolated weak cells mark model-specific failures against otherwise tractable evidence. The row corresponding to starcoder2:7b stands out as uniformly low throughout.

IX. Discussion

A. RQ A — Trustworthiness Score

Hallucination Frequency turns out to be statistically independent from the other three metrics (r 0.0 with TA, ECA, and ASC). In practice, this means a model can hallucinate fairly often while still producing accurate findings when it

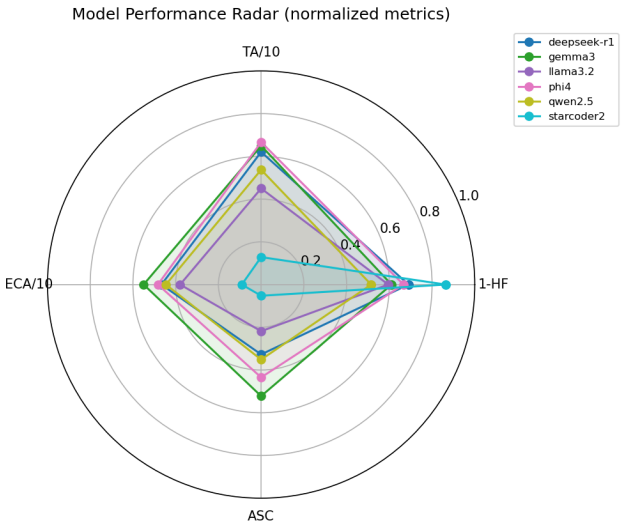

Fig. 5. Model performance radar over the four normalised metrics.

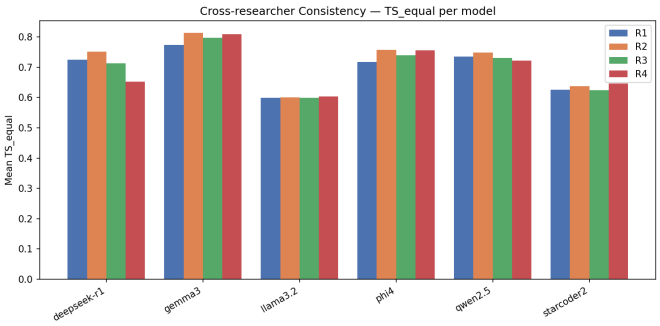

Fig. 6. Mean TS_{equal} per model, broken down by callsite batch.

does flag something, and the reverse is equally true. PCA backs this up: PC1 captures 64% of variance and loads almost entirely on TA, ECA, and ASC, while HF lands on PC2. Simply averaging all four metrics equally glosses over this two-dimensional structure.

The most defensible approach to a trustworthiness score is to treat HF on its own terms: one composite for "quality when something is found" (TA + ECA + ASC), with HF handled as a separate penalty. That said, the more pressing issue is inter-auditor agreement. ICC values near zero (-0.10 to 0.11) indicate auditors disagree substantially on individual callsites. This doesn't sink the metrics, but it does mean individual audit scores are noisy. Only aggregated means across multiple auditors are solid enough to cite with confidence.

B. RQ B — Model Benchmarking

Gemma3:12b comes out on top, with phi4:14b, deepseek-r1:8b, and qwen2.5-coder:7b forming a cluster just behind it that is statistically indistinguishable from one another. At the bottom, llama3.2:3b and starcoder2:7b are clearly worse than the leading group (Bonferroni-corrected p ¡ 0.001). The notable surprise here is qwen2.5-coder, the only specialist code model in the set, finishing fourth behind two general-purpose models. That result suggests security reasoning across a Java-JS bridge draws more on broad language understanding than on code-specific training. Parameter count also seems to matter: the 12b and 14b models dominate, while the 3b model struggles noticeably. Starcoder2's weak ECA score (mean 0.90 vs. gemma3's 5.50) points to a consistent blind spot in attack chain reasoning. It catches individual issues but tends to miss how they connect downstream.

C. RQ C — Failure Taxonomy

Context window size does not appear to be the main factor driving failures. Context length correlates with performance at r ¡ 0.13 across all models, and if anything the relationship nudges slightly positive: richer, longer contexts weakly help rather than hurt. The same holds for bridge complexity. More exposed methods actually correlates with marginally better trustworthiness scores (p ¡ 0.01), which suggests that more surface area gives models more to anchor their reasoning to. The real failure modes are elsewhere: hallucination instability and chain reasoning collapse. Llama3.2 hallucinates in 15.9% of audits, nearly three times gemma3's rate, and this doesn't track with context length since hallucination spikes tend to occur in shorter contexts (mean 408 chars vs. 512 for normal audits). The pattern points to low-parameter models confabulating findings when the bridge interface gives them little to work with. Starcoder2 fails differently: its hallucination rate is low, but its effect chain awareness is consistently poor. It surfaces issues at the symptom level but struggles to reason through multi-step exploitation paths. These are meaningfully distinct failure modes and worth keeping separate in the taxonomy: confabulation failure (llama3.2) versus shallow reasoning failure (starcoder2).

X. Conclusion

This paper set out to ask not whether large language models can produce plausible-sounding security analysis of Android WebView bridges, but whether their output can be trusted enough to act on. The Trust, but Verify framework operationalized that question by separating analysis from verification across distinct model instances and quantifying the result through four metrics combined into a Trustworthiness Score. The findings do not support a confident answer in either direction, and that ambiguity is itself the central result.

Among the six models tested, gemma3:12b emerged as the strongest performer, with phi4:14b, deepseek-r1:8b, and qwen2.5-coder:7b forming a statistically indistinguishable second tier. The lower bound was occupied by llama3.2:3b and starcoder2:7b, which failed in qualitatively different ways: the former through confabulation, the latter through shallow chain reasoning. The strongest specialist code model finished behind two general-purpose ones, suggesting that reasoning across

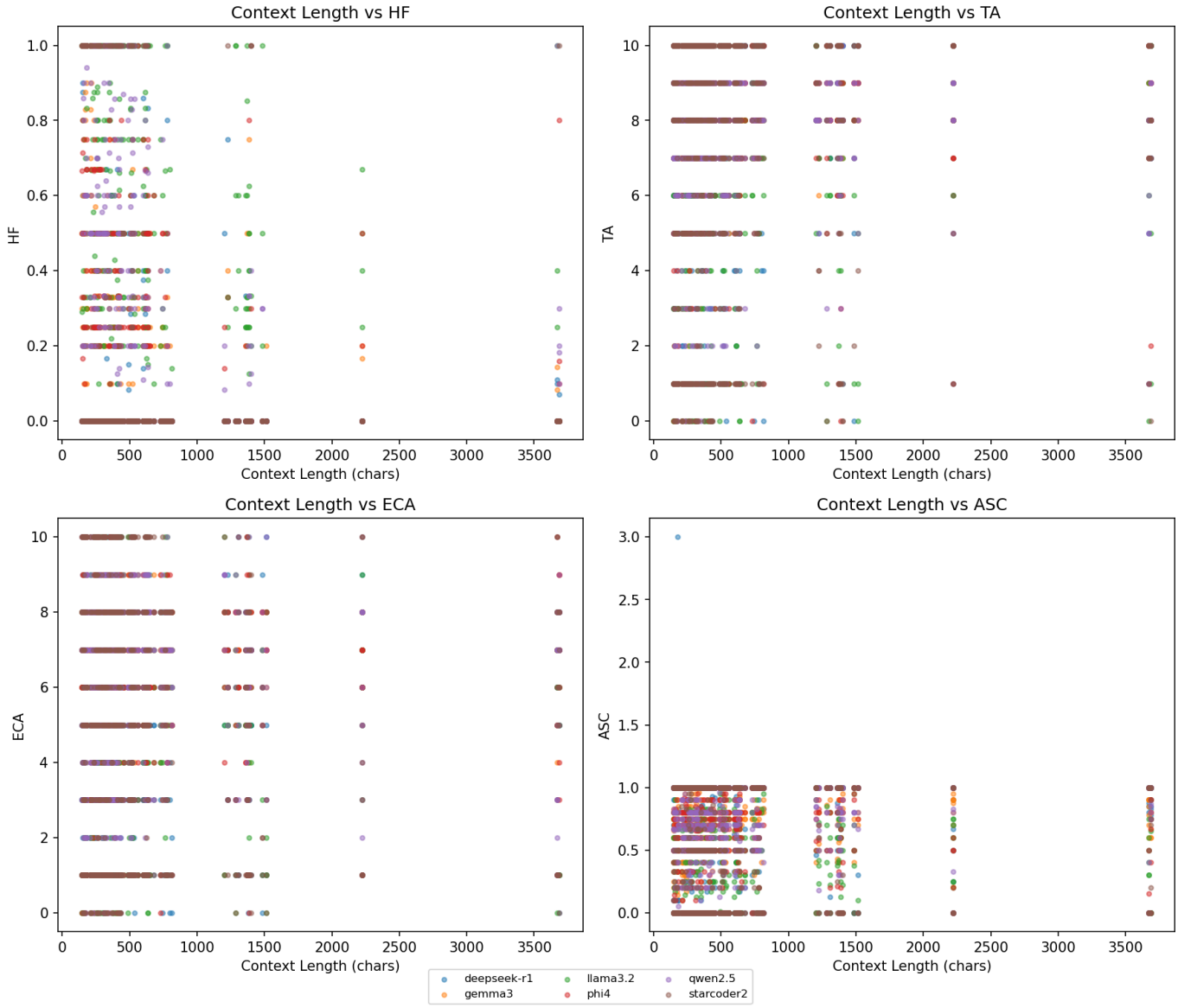

Fig. 7. Each of the four metrics plotted against per-audit context length.

the Java–JavaScript boundary draws more on broad semantic understanding than on code-specific training.

Two caveats temper any optimistic reading. Inter-auditor agreement was poor, with ICC values near zero, meaning individual audit scores are too noisy to support claims about specific findings on specific callsites; only aggregated means across multiple auditors are stable enough to cite. Hallucination frequency was also statistically independent from the other three quality dimensions, meaning a model can be technically accurate and contextually aware while still fabricating findings at a measurable rate. Taken together, these phenomena undermine the premise that LLM-generated security findings can be treated as reliable on their own. The models tested are not suitable for autonomous use in a security pipeline, and the audit layer, while useful for quantifying degradation, does not substitute for independent human verification. The contribution of this work is therefore methodological: a means of measuring how far short of trustworthy a given model falls, applied to a population in which all members fall short.

XI. Perspective

Trying to quantify a concept like trustworthiness comes with inherent problems, most notably the very definition of someone, or something, being ”trustworthy”. When it comes to a model control or further agency, different purposes may warrant varying priorities in terms of preferring false positives or false negatives. Given the results observed, it’s clear that no model can ever be given any variant of responsibility, however, there may exist cases for which the observed behavior is ”good enough”. For instance, if there exists a model that cannot

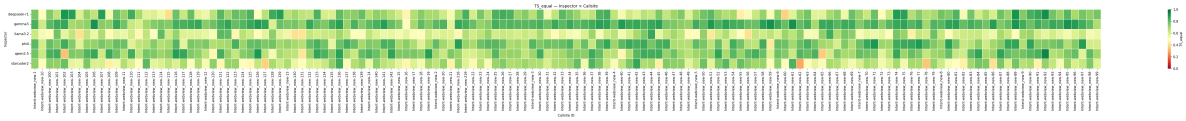

Fig. 8. TS_{equal} per model (rows) per callsite (columns), spanning the full 145-callsite population.

be proven to provide false-negatives, such a model may be useful for projects where safety is a priority. The rate of false-negatives to true-negatives is going to determine how useful such a model can be, but if a model could be used to reduce the workload by just 10% by screening submitted issues or the like, that might be of benefit. However, that is a momentous burden of proof, and given the performance of the models tested, it does not appear likely among similar models.

A. Expanding the Testing Sample

Repeating the experiments conducted with more models of both lower and higher parameter counts could be of interest. There may be some local maximum, given that the largest model tested in the current population was 14 billion, yet did not end up with the best results. Massive models like Claude Opus 4.7 (estimated parameter count of 1-10 trillion, however no source found), but even smaller models like ChatGPT 3.5 (reportedly 20 billion parameters) could provide interesting results.

B. Dataset Choice

There exist several bug benchmark datasets which may prove vastly more optimal for the type of evaluation conducted in this paper. For instance, SWE-bench Verified would allow the testing procedure to skip the time consuming researcher evaluation step, given that all bugs in the dataset are human verified and thus known. That being said, the very concise security perspective regarding Android Webview apps would be lost, given that SWE-bench Verified has nothing inherently to do with security. However, it could still provide valuable data on the model's ability to audit their own or others analysis of code.

References

[1] J. Prakash, A. Tiwari, S. Groß, and C. Hammer, "LUDroid: A large scale analysis of Android–Web hybridization," in *Proceedings of the IEEE/ACM International Conference on Program Comprehension*, 2019. University of Potsdam.

[2] J. Prakash, A. Tiwari, and C. Hammer, "iWanDroid: Demand-driven information flow analysis of WebView in Android hybrid apps." The 34th IEEE International Symposium on Software Reliability Engineering (ISSRE), Florence, 2023. Dataset, version 1.0.

[3] Z. Sheng, Z. Chen, S. Gu, H. Huang, G. Gu, and J. Huang, "LLMs in software security: A survey of vulnerability detection techniques and insights," *ACM Computing Surveys*, 2025. arXiv:2502.07049.

[4] T. Liang, Z. He, W. Jiao, X. Wang, Y. Wang, R. Wang, Y. Yang, S. Shi, and Z. Tu, "Encouraging divergent thinking in large language models through multi-agent debate," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 17889–17904, Association for Computational Linguistics, 2024.

[5] S. Hu, T. Huang, F. Ilhan, S. F. Tekin, and L. Liu, "Large language model-powered smart contract vulnerability detection: New perspectives," in *IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS)*, 2023.

[6] Android Developers, "Build web apps in webview," 2026. Last updated 2026-05-20 UTC.

[7] S. Vatsal and H. Dubey, "A survey of prompt engineering methods in large language models for different NLP tasks," 2024.

[8] K. Sun and R. Wang, "Breaking myths in llm scaling and emergent abilities with a comprehensive statistical analysis," *Neurocomputing*, vol. 670, p. 132542, 2026.

XII. Appendix

A. Source Code

The source code can be reached at:
https://github.com/Engineering-Research-in-Software/llm-static-analysis

B. Result Sets

The results set can be found at:
https://github.com/Engineering-Research-in-Software/llm-static-analysis/tree/feature/firstLLMAnalyzerPOC/analyzer/validated
and
https://github.com/Engineering-Research-in-Software/llm-static-analysis/tree/feature/firstLLMAnalyzerPOC/analyzer/src/research

C. Dataset

The prepared dataset can be found at:
https://github.com/Engineering-Research-in-Software/llm-static-analysis/blob/feature/firstLLMAnalyzerPOC/analyzer/results/prepared_dataset.json

TABLE II
GROUP MEMBERS AND CONTRIBUTION AREAS

Name	Email	Contribution Area
João R. Cardoso	jocar25@student.sdu.dk	Metric Ideation and Definition, Prompt-Engineering, System Architecture, Manual Result Validation
Lily B. Wanscher	liwan21@student.sdu.dk	Metric Ideation and Definition, System Integrations, Test Suite Execution, Resultset Processing, Manual Result Validation
Denisa A. Rissa	deris22@student.sdu.dk	Metric Ideation and Definition, Manual Result Validation
Yusuf M. Yusuf	yuyus19@student.sdu.dk	Metric Ideation and Definition, LLM Model List, Manual Result Validation

All members mentioned above have contributed in equal importance toward the development of this project.